

Pre-trained 모델 활용

03 이미지 생성 모델

목차

—
그림을 그리는 생성 AI
모델인 Stable Diffusion을
사용하는 과정을 배웁니다.

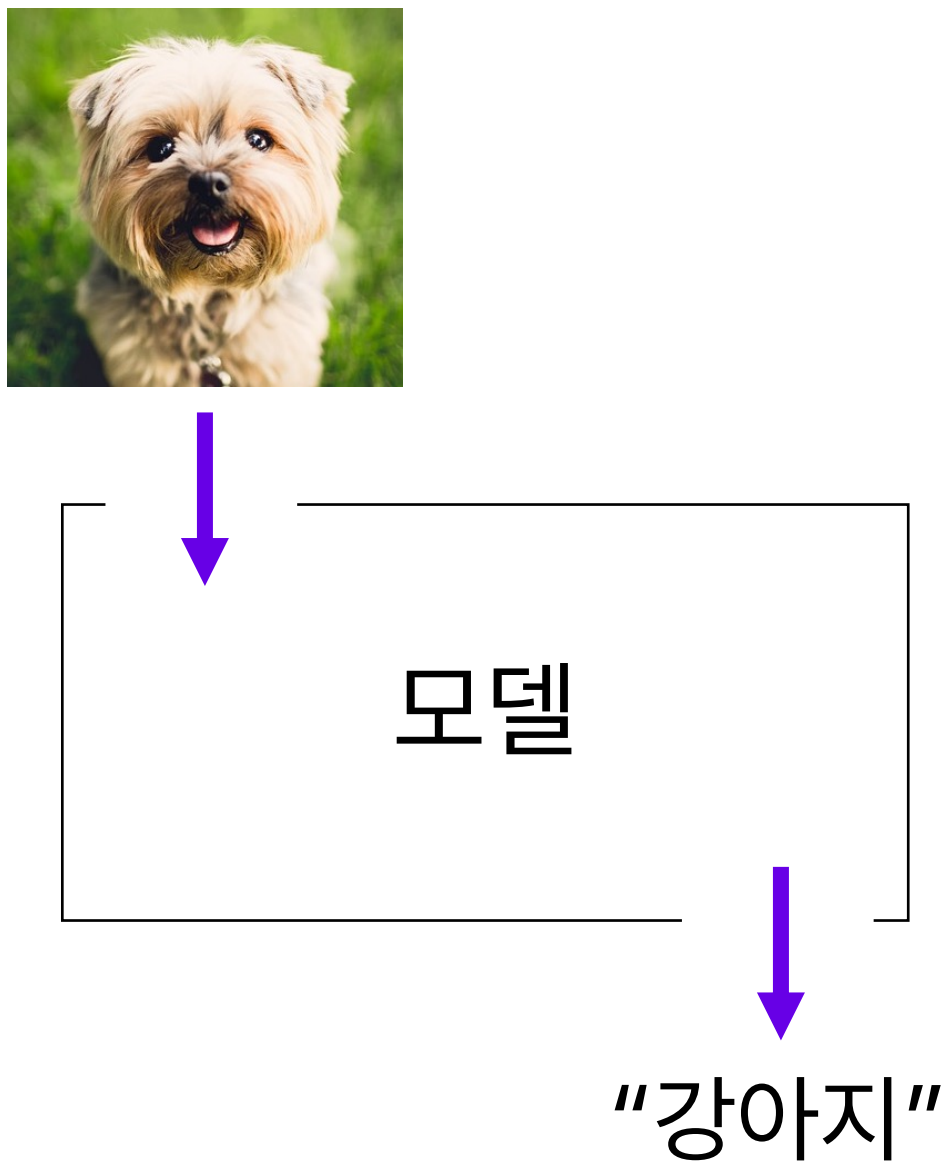
- 01 **Stable Diffusion**
- 02 **이미지 생성하기**
- 03 **파이프라인 옵션**

01

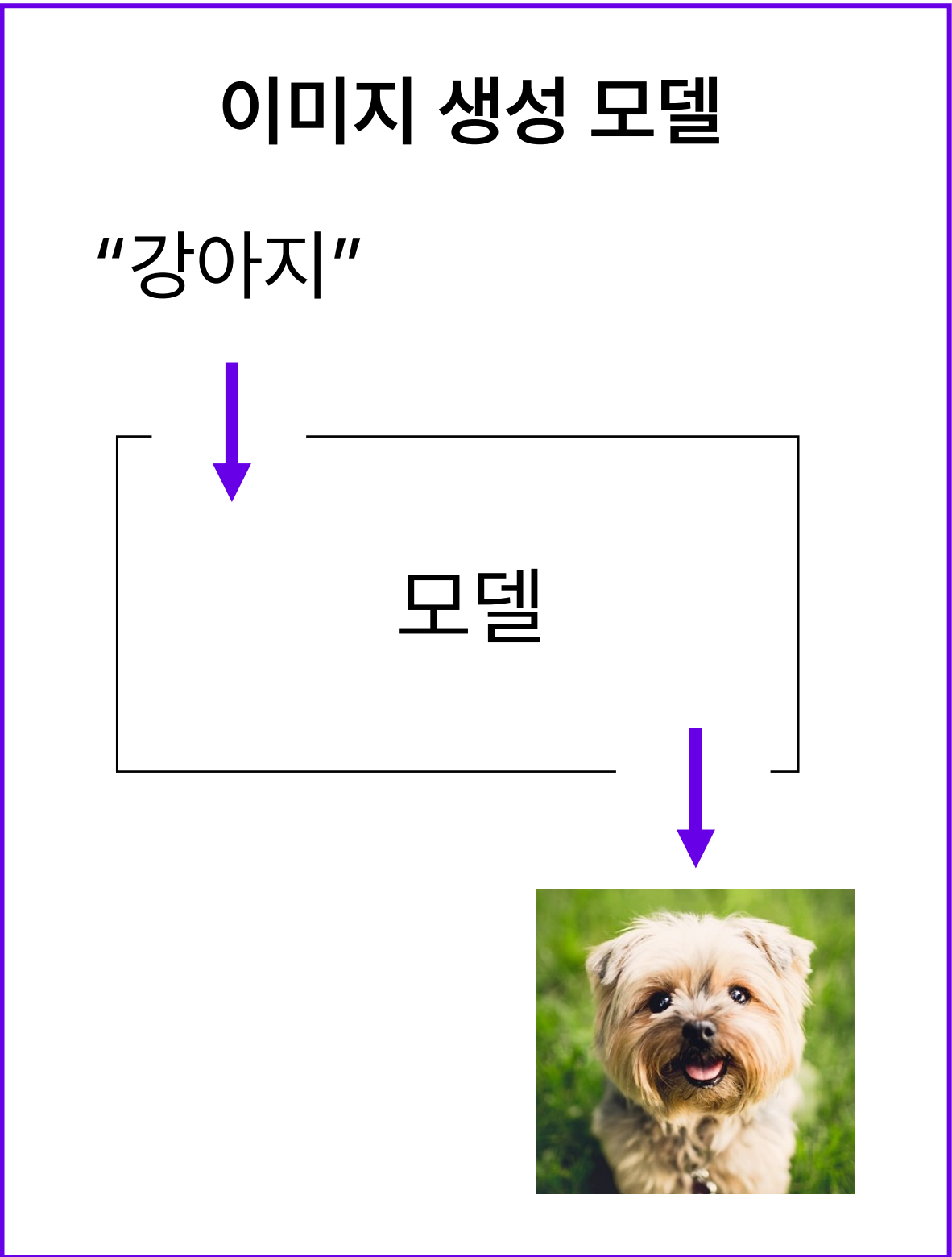
Stable Diffusion

👍 이미지 생성 모델

이미지 분류 모델



이미지 생성 모델



☑ Stable Diffusion이란?

이미지 생성 모델의 일종으로 높은 품질의 이미지를 생성할 수 있음

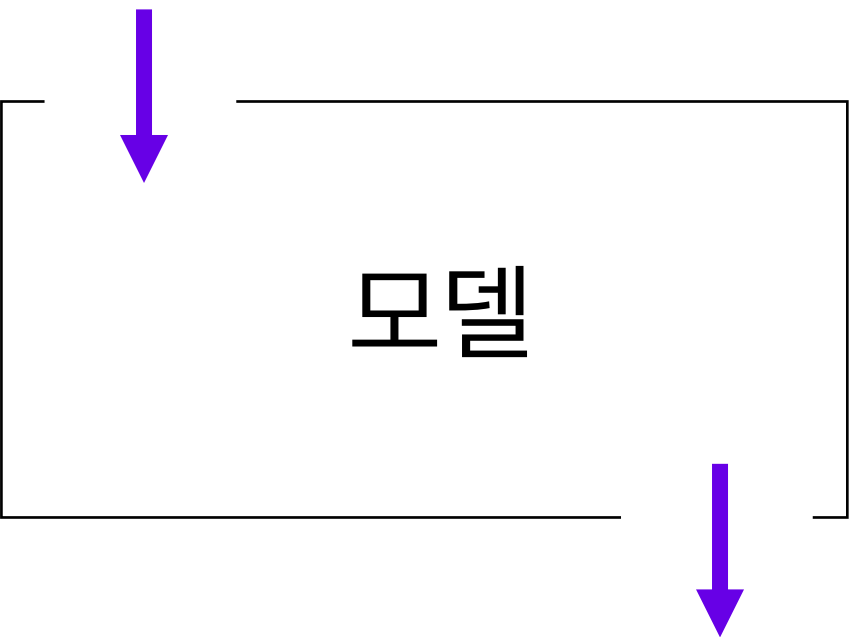
- CVPR 2022에 발표된 논문을 기반한 모델.^[1]
- 상대적으로 적은 자원으로 높은 품질의 이미지를 생성할 수 있게 되면서 폭발적인 인기를 끌.

[1] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 10684-10695).

④ 프롬프트(Prompt)

이미지 생성 모델

"강아지" 프롬프트(Prompt): 모델에 입력하는 자연어 데이터



④ 파이프라인

모델을 사용하는 여러가지 과정을 하나로 묶어서 처리

- 자연어 데이터를 입력하면 이미지를 만들기까지의 과정을 알아서 처리함.
- 기본으로 제공되는 파이프라인만 이용해도 이미지를 만들 수 있음.

④ 모델의 동작 원리- Stable Diffusion

무작위의 노이즈에서 시작해 프롬프트에 맞는 이미지를 생성하는 모델

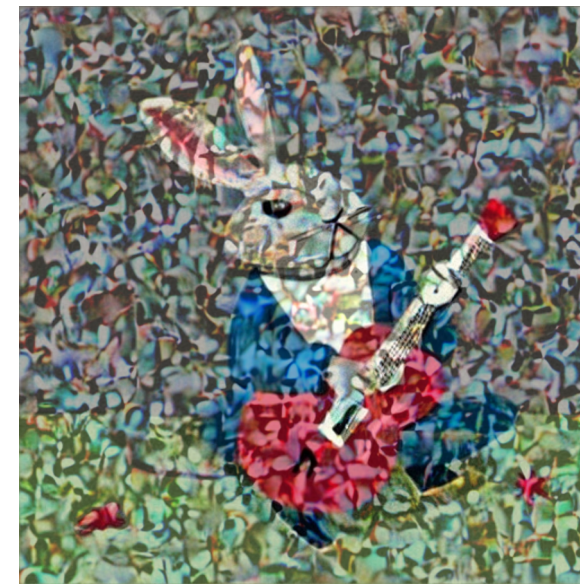
- 온전한 이미지에 노이즈를 추가하면서 학습을 진행.
- 학습이 완료된 모델은 노이즈를 보고 프롬프트에 맞는 이미지를 복원하는 모델이 됨.

☑ Stable Diffusion 학습 과정

노이즈를 추가하고 힌트와 함께 원본을 복구하는 학습을 진행



노이즈 추가



01 Stable Diffusion

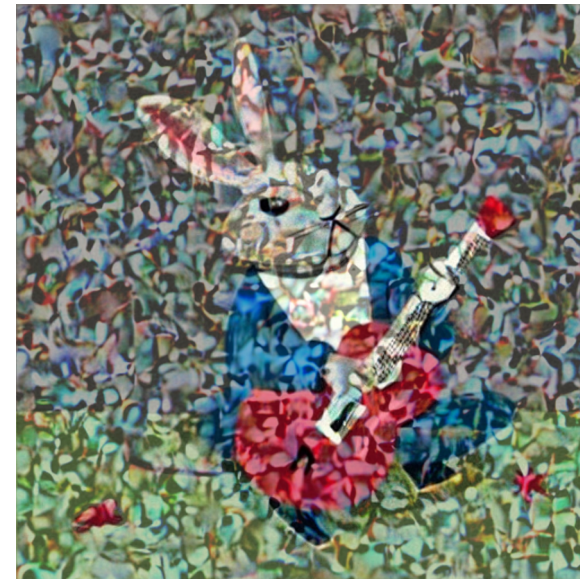
☑ Stable Diffusion 학습 과정

노이즈를 추가하고 힌트와 함께 원본을 복구하는 학습을 진행

Prompt

이 이미지는 원래 "기타를 치는 토끼 그림"인데
거기에 노이즈를 추가한 결과야

모델



이미지 복원

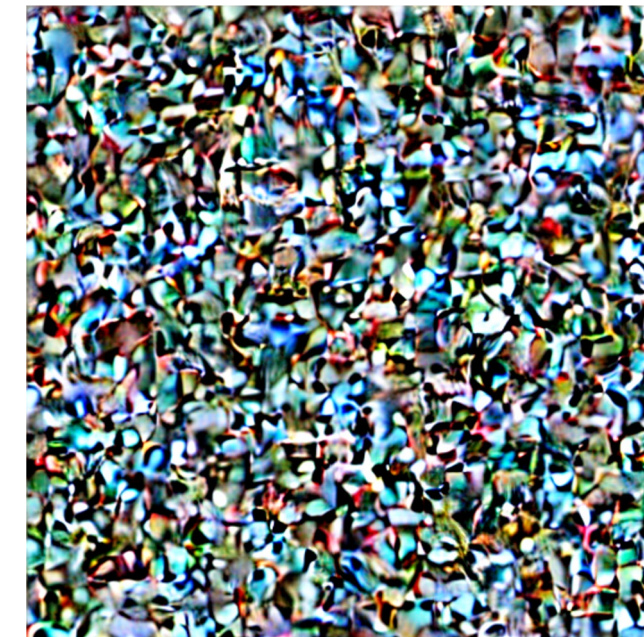


④ 학습이 끝난 Stable Diffusion 모델

힌트를 통해 노이즈를 이미지로 복원하는 과정을 반복

이 이미지는 원래 "기타를 치는 토끼 그림"인데
거기에 노이즈를 추가한 결과야

모델



01 Stable Diffusion

④ 학습이 끝난 Stable Diffusion 모델

힌트를 통해 노이즈를 이미지로 복원하는 과정을 반복

이 이미지는 원래 "기타를 치는 토끼 그림"인데
거기에 노이즈를 추가한 결과야

모델

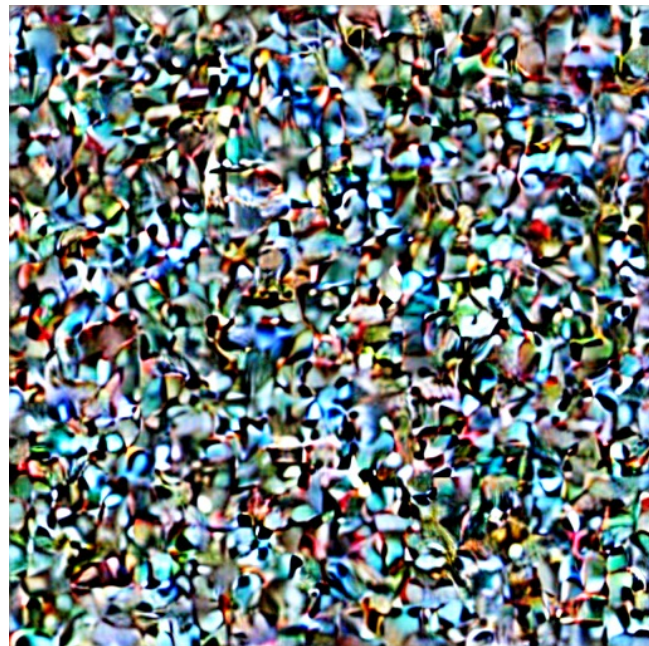


01 Stable Diffusion

☑ 이미지 생성 Stable Diffusion 모델

Steps

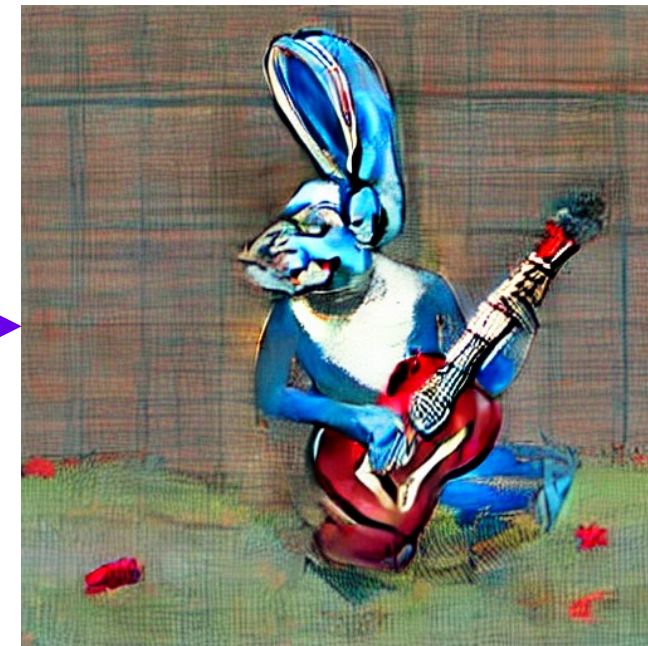
노이즈를 이미지로 복원하는 작업을 **N번** 반복



1번 반복



3번 반복



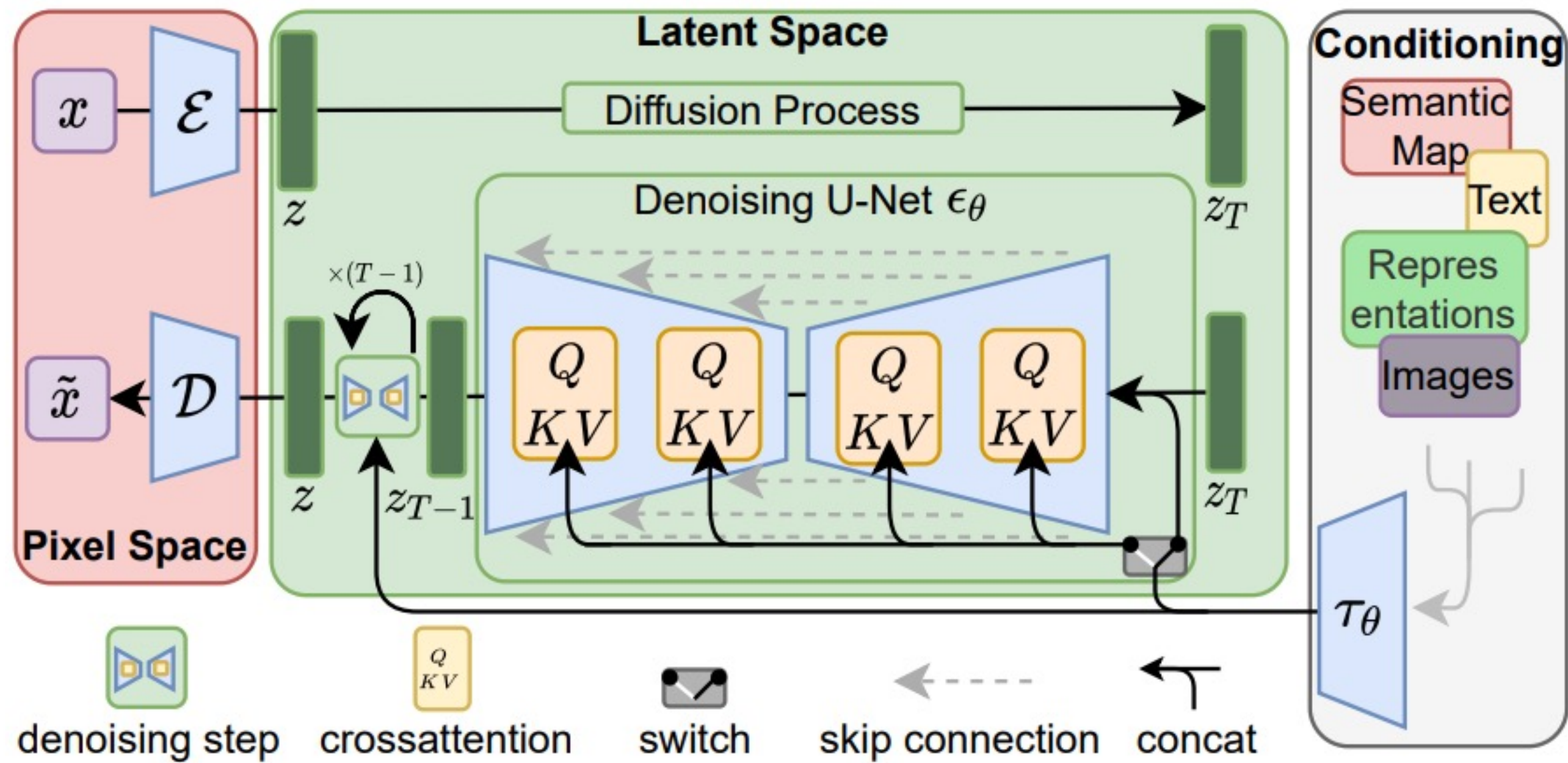
5번 반복



40번 반복

01 Stable Diffusion

☑ 전체 모델 구조



02

이미지 생성하기

02 이미지 생성하기

④ 파이프라인 불러오기

```
import torch # 딥러닝 라이브러리
import PIL # 이미지 처리 라이브러리
from diffusers import StableDiffusionPipeline # 파이프라인 객체
```

- Stable Diffusion 파이프라인을 불러옴

02 이미지 생성하기

☑ 사전 학습된 모델 불러오기

```
pipe = StableDiffusionPipeline.from_pretrained("CompVis/stable-diffusion-v1-4", torch_dtype=torch.float16)
```

- from_pretrained 메서드를 이용해 사전학습된 모델을 불러옴
- 아래 경로의 모델을 불러와 파이프라인에서 사용
 - <https://huggingface.co/CompVis/stable-diffusion-v1-4>

02 이미지 생성하기

☑ GPU 사용하기

```
pipe = pipe.to("cuda")
```

- 파이프라인이 GPU를 사용하여 연산을 빠르게 할 수 있도록 CUDA GPU로 이동

02 이미지 생성하기

④ 이미지 생성하기

```
prompt = "a rabbit playing guitar"  
images = pipe(prompt).images #결과에서 이미지들을 가져옴  
images[0] # 생성한 이미지 객체
```

- 다른 옵션은 모두 기본값으로 설정하고 생성.
- 파이프라인에 프롬프트를 전달하면 이미지가 생성됨.
- 하나의 이미지만 생성했기에 0번째 이미지를 가져옴.



02 이미지 생성하기

👉 파일로 저장하기

```
image.save(f"{prompt}.png") # 생성된 이미지를 파일로 저장
```

 a rabbit playing guitar.png

- PIL의 Image 객체이므로 save함수를 이용해 저장.

02 이미지 생성하기

④ 매번 다른 이미지

무작위의 노이즈에서 시작하기 때문에 처음 노이즈에 따라 다른 그림이 생성



④ 동일한 이미지를 얻는 방법

노이즈를 생성할 때 시드가 동일하면 같은 노이즈에서 시작

- 다른 조건은 고정하고 옵션을 변경하기 위해서 시드를 고정해야 할 필요가 있음.
- 시드, 프롬프트, 옵션이 모두 동일하면 같은 이미지가 생성됨.

02 이미지 생성하기

☑ 시드 고정하기

```
generator = torch.Generator("cuda").manual_seed( 1234 ) #시드를 1234로 설정  
image = pipe(prompt, generator=generator).images[0] # 이미지 생성
```

- 무작위의 값을 생성하는 제너레이터 객체를 만들어 파이프라인에 전달.
- manual_seed 함수에 원하는 값을 전달할 수 있음.
- 생성된 객체를 파이프라인에 전달하면 해당 시드를 사용해 이미지를 생성함.

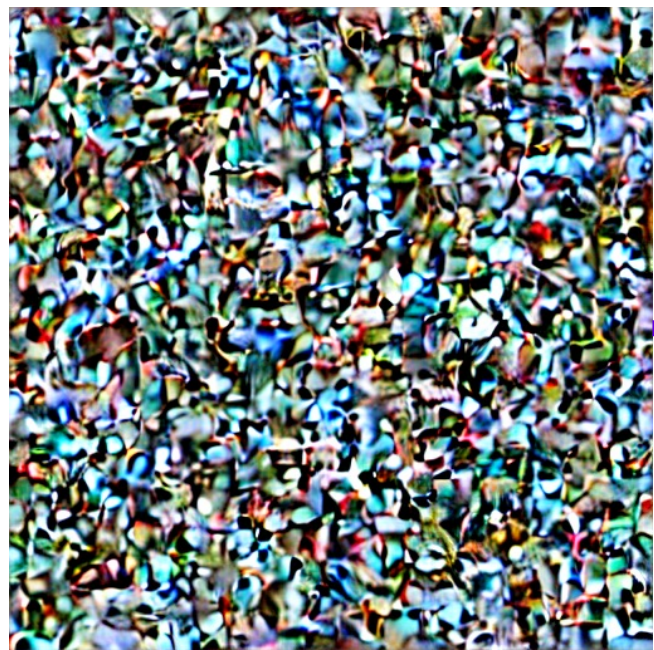


03

파이프라인 옵션

☑ Step

노이즈를 이미지로 복원하는 작업을 반복하는 횟수



1번 반복
step = 1



3번 반복
step = 3



5번 반복
step = 5



40번 반복
step = 40

03 파이프라인 옵션

☑ Step

```
generator = torch.Generator("cuda").manual_seed(1234) #시드를 1234로 설정  
image = pipe(prompt, num_inference_steps=30, generator=generator).images[0] # 30단계 동안 생성
```

- num_inference_steps 매개변수를 통해 설정.
- 클수록 선명한 이미지를 생성하지만, 항상 보기 좋게 완성되는 것은 아님.
- 너무 작은 수를 전달하면 노이즈에 가깝거나 미완성의 그림이 생성됨.
- 여러 값을 실험하면서 최적의 값을 찾는 과정이 필요함.



☑ Step

여러 값을 실험하면서 적절한 값을 찾는 과정이 필요



step = 30



step = 50



step = 70

④ guidance scale

이미지가 얼마나 프롬프트와 강하게 연결될 것인지를 결정

- 기본값은 7.5
- 이 값이 크면 품질을 희생하더라도 프롬프트와의 연결성을 강하게 만듦.
- 이 값이 작으면 프롬프트와 관련 없는 그림을 생성할 가능성이 증가함.

03 파이프라인 옵션

👉 guidance scale

```
generator = torch.Generator("cuda").manual_seed(1234) #시드를 1234로 설정  
image = pipe(prompt, generator=generator, guidance_scale=10).images[0] # guidance_scale을 10으로 설정
```

- guidance_scale 매개변수를 통해 전달.
- 전달하지 않으면 7.5가 적용됨.



03 파이프라인 옵션

☑ guidance scale

모든 옵션은 여러 값을 실험하면서 적절한 값을 찾는 과정이 필요



scale = 1



scale = 7.5



scale = 10

④ 이미지 크기 변경

생성되는 이미지의 크기를 변경

- height 와 width 매개변수를 통해 설정함.
- 모두 8의 배수로 설정되어야 함.
- 기본값은 512x512.
- 사이즈가 너무 크면 GPU의 메모리가 넘치면서 오류가 발생함.

03 파이프라인 옵션

④ 이미지 크기 변경

```
image = pipe(prompt, generator=generator, height=768, width=512).images[0]
```

- height 와 width 매개변수를 통해 설정함.
- 모두 8의 배수로 설정되어야 함.



03 파이프라인 옵션

☑ 이미지 크기와 시드

시드가 같아도 크기가 달라지면 전혀 다른 이미지가 생성됨



seed=1234
width=512
height=768



seed=1234
width=512
height=512



seed=1234
width=760
height=512

④ 도우미 함수

각종 옵션과 프롬프트를 전달하면 이미지를 생성하여 반환하는 함수

- 잘 생성된 그림의 시드를 기억하기 위해 적용된 시드를 출력.
- 생성된 이미지 객체를 반환하는 함수.

03 파이프라인 옵션

④ 도우미 함수 코드

```
def generate_image(
    prompt: str,
    width: int = 512,
    height: int = 512,
    seed: int = -1,
    guidance_scale: float = 7.5,
    steps: int = 50,
) -> PIL.Image.Image:
    if seed == -1:
        seed = random.randint(0, 2**32)
    generator = torch.Generator("cuda").manual_seed(seed)
    image = pipe(
        prompt,
        guidance_scale=guidance_scale,
        generator=generator,
        width=width,
        height=height,
        num_inference_steps=steps,
    ).images[0]

    print(f"seed: {seed}")
    return image
```

- prompt (str): 생성할 이미지의 자연어 프롬프트
- width (int, optional): 이미지의 가로 (8의 배수). 기본값은 512.
- height (int, optional): 이미지의 높이 (8의 배수). 기본값은 512.
- seed (int, optional): 시드. -1을 전달하면 무작위 시드가 배정.
- guidance_scale (float, optional): 이미지가 프롬프트에 가깝게 나오는 정도. 기본값은 7.5.
- steps (int, optional): num_inference_steps. 이미지를 생성하는 단계의 수. 기본값은 50.

03 파이프라인 옵션

④ 도우미 함수를 이용한 이미지 생성

```
image = generate_image(  
    "Tiny cute rabbit toy, reading a book, standing character, square image",  
    seed=4247736899,  
)  
image
```

- 시드가 출력되고 이미지를 반환.

